Holistic Deployment Strategy: From Code to Production and Back

Team Introduction & Overview

What This Is Really About

This is **not just about versioning**. This is about establishing a **complete, reliable deployment ecosystem** that connects every step from writing code to deploying to production, and critically, **rolling back when things go wrong**.

We're defining how:

- **Git branching** controls what gets deployed when
- **CI/CD pipelines** automate building, testing, and releasing
- **Semantic versioning** provides clear communication about changes
- **Artifact storage** preserves every release for instant rollback
- **Deployment processes** move code safely through environments
- **Version tracking** shows what's running where
- **Rollback strategies** recover from problems in minutes, not hours

These are not separate topics. They are **interconnected parts of a single system** that determines how fast we can ship, how confident we can be, and how quickly we can recover.

Why This Matters to Everyone

For Developers

- **Clarity**: Know exactly when your code gets deployed
- **Safety**: Merge with confidence - pipelines catch issues before production
- **Speed**: No manual deployment coordination
- **Recovery**: When something breaks, rollback is automated and fast

For DevOps

- **Automation**: Pipelines handle the heavy lifting
- **Traceability**: Know exactly what's deployed where
- **Reliability**: Consistent process reduces human error
- **Control**: Clear rules everyone follows

For The Business

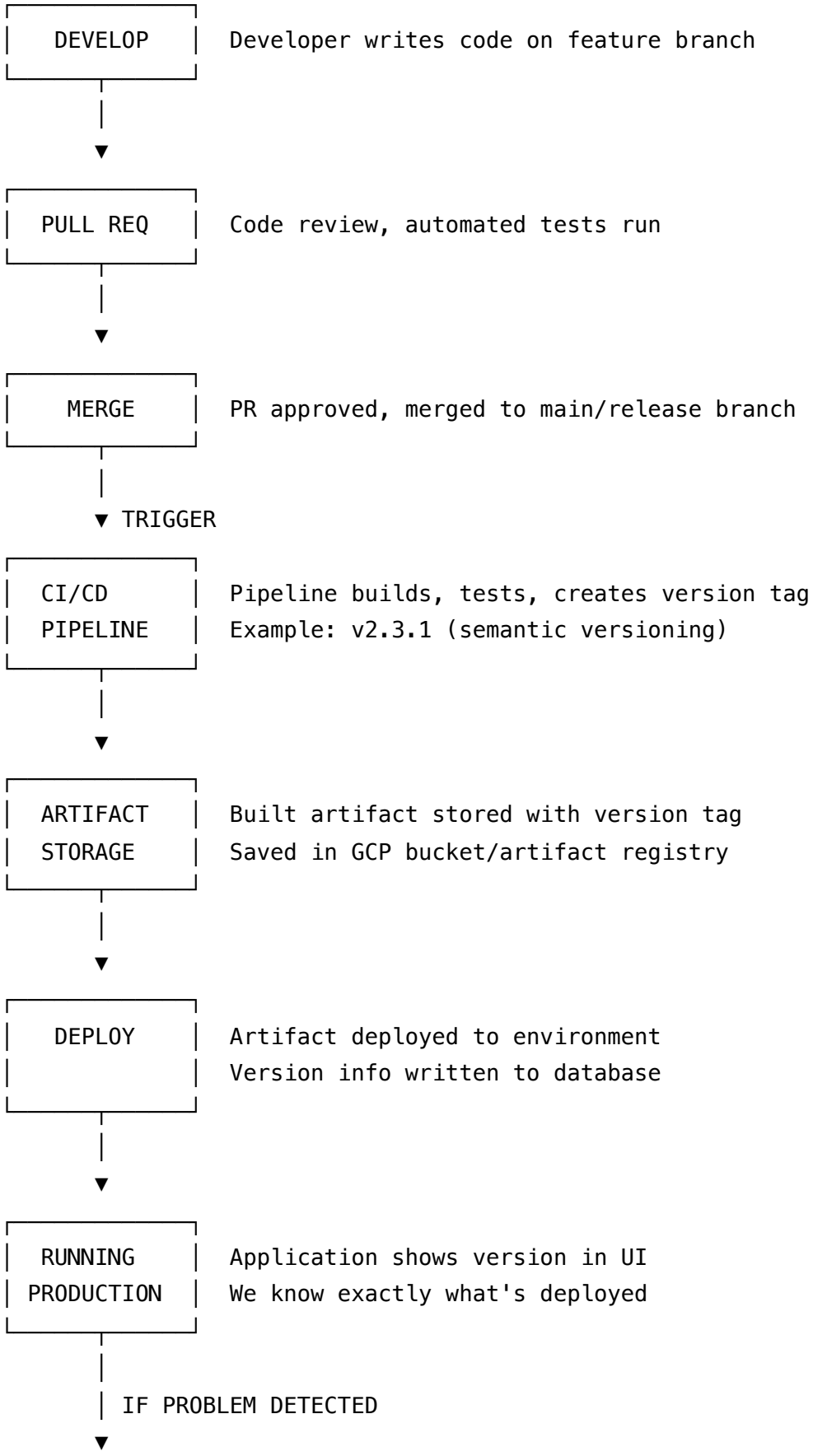
- **Faster releases**: Automation speeds up time-to-market
- **Lower risk**: Automated testing and easy rollbacks reduce deployment fear
- **Better visibility**: Always know what version is running
- **Cost savings**: Less time firefighting production issues

The Cost of NOT Having This

Without a holistic strategy:

- Deployments become manual, error-prone, slow
- Rollbacks are painful and risky ("which version was working?")
- Developers hesitate to merge ("what if it breaks?")
- Production issues take hours to diagnose ("what changed?")
- Inconsistent versions across services cause mysterious bugs

The Complete Flow: How It All Connects



```
┌──────────────┐
│  ROLLBACK    │  Deploy previous version artifact
│              │  Minutes, not hours
└──────────────┘
```

Key Insight: Each step depends on the previous one. Break one link and the whole chain fails.

Core Principles

1. One Source of Truth: Git

- Git branches control the deployment lifecycle
- Tags represent deployable versions
- Main/release branches are always deployable
- Feature branches never get deployed directly

2. Automation Over Manual Process

- Pipelines handle building, testing, tagging, deploying
- Humans write code and approve merges
- Everything else is automated
- No manual version bumping or artifact creation

3. Semantic Versioning Everywhere

- `MAJOR.MINOR.PATCH` format consistently across all services
- MAJOR = breaking changes
- MINOR = new features (backward compatible)
- PATCH = bug fixes
- Every service uses the same convention

4. Every Version Is Recoverable

- Every version stored as an artifact
- Artifacts immutable (never modified)
- Rollback = deploy previous artifact
- No "recreating" or "remembering" what was deployed

5. System-Level Thinking

- Individual service versions (Backend v1.5.2, Frontend v2.1.0)
- System version groups them (System v2.3.1)
- Deploy and rollback at system level for customer environments
- Know what combination of services is tested and working

6. Version Information Is Deployment Metadata

- Versions recorded during deployment (not hardcoded)
- Stored in database at deployment time
- Frontend queries dynamically
- Always accurate for each environment

What Changes for Our Team?

Developers Must:

1. **Follow branching strategy** - feature branches, PRs to main
2. **Never bypass the process** - no direct merges, no manual deploys
3. **Trust the pipelines** - they're designed to catch issues
4. **Understand semantic versioning** - know when changes require major vs minor bump

DevOps Must:

1. **Set up and maintain pipelines** - ensure they're reliable
2. **Configure artifact storage** - for all components
3. **Implement version tracking** - database table, API endpoint
4. **Document the process** - keep it clear and up-to-date

Everyone Must:

1. **Follow ONE agreed convention** - inconsistency breaks everything
2. **Communicate about breaking changes** - coordinate major version bumps
3. **Use the version display** - check what's deployed before debugging
4. **Participate in continuous improvement** - process evolves based on learning

Our Multi-Service Architecture

We have multiple components that must work together:

Component	Repository	Version Independently
TMS Database	tms-alloydb-schema	Yes - schema has its own lifecycle
TMS Bridge	Disposition-Abstraction-Layer	Yes - API between New Dispo and TMS
Backend	Disposition-Backend	Yes - core business logic
Frontend	Disposition-Frontend	Yes - UI changes frequently
Cloud Functions	CALConsult.Disposition.Functions	Yes - serverless components
Cloud4Log	Cloud4Log	Yes - separate logging system

Challenge: Coordination

- Each component versions independently
- But they must work together
- **Solution**: System versioning groups tested combinations
- Customer deployments use system versions for stability

The Path Forward

Immediate Next Steps

1. **Align on conventions** - One branching strategy, one tagging strategy
2. **Document everything** - So everyone can reference it
3. **Set up infrastructure** - Artifact storage, version database table
4. **Configure pipelines** - Automate the flow
5. **Train the team** - Ensure everyone understands their role
6. **Start with one service** - Prove the pattern, then expand

Long-Term Benefits

- **Faster development cycles** - Confidence to merge and deploy frequently
- **Reduced production incidents** - Better testing, easier rollback
- **Clear communication** - Everyone knows what's deployed where
- **Scalable process** - Works as team and codebase grow
- **Professional operation** - Industry best practices

Next: The Detailed Technical Guide

This overview explains the **what and why**. For the **how**, see:

Best Practices: Versioning, Branching & Version Display

That document contains:

- Detailed semantic versioning rules
- Branching strategy specifics
- Pipeline design patterns
- Database schema for version tracking
- Implementation checklist
- Code examples

Questions to Discuss

As we move forward, we need team input on:

1. **Branching strategy details** - What fits our workflow best?
2. **Version bump triggers** - When do we increment major vs minor vs patch?
3. **Deployment frequency** - How often do we want to release?
4. **Rollback procedures** - Who can trigger a rollback and how?
5. **Testing requirements** - What must pass before artifact creation?
6. **Communication protocols** - How do we coordinate breaking changes?

Your Role

This is a **team effort**:

- **Developers**: Follow the workflow, provide feedback on pain points
- **DevOps (Gojko)**: Lead implementation, maintain infrastructure
- **Matthias**: Overall coordination, decision facilitation
- **Everyone**: Hold each other accountable to the agreed process

Success requires everyone's commitment. The process only works if we **all follow it consistently**.

Remember: This is not about making life harder. It's about making deployment **safer, faster, and more reliable**. The upfront investment in process pays dividends every single day.

Based on consultation with DevOps expert Marko (2026-02-25)